

MATH 342W / 642 / RM 742 Spring 2024 HW #5

Bryan Nevarez

Wednesday 22nd May, 2024

Problem 1

These are some questions related to probability estimation modeling and asymmetric cost modeling.

- (a) [easy] Why is logistic regression an example of a “generalized linear model” (glm)?

A logistic regression is an example of a “generalized linear model” due to:

- the ‘signal’ $s = \mathbf{w} \cdot \mathbf{x}$ has the input variables as a linear combination
- the response variable, y_i , are independent observations that come from a common probability distribution
- a link function (*referred to in part(b) below*)

- (b) [easy] What is $\mathcal{H}_{pr}$ for the probability estimation algorithm that employs the linear model in the covariates with logistic link function?

For the probability estimation algorithm that employs the linear model in the covariates with a logistic link function, ϕ , we have,

$$\mathcal{H}_{pr} = \left\{ \phi(\mathbf{x}, \mathbf{w}) = \frac{1}{1 + e^{-\mathbf{x} \cdot \mathbf{w}}} : \mathbf{w} \in \mathbb{R}^{p+1} \right\}.$$

- (c) [easy] If logistic regression predicts 3.1415 for a new $\mathbf{x}_*$, what is the probability estimate that $y = 1$ for this $\mathbf{x}_*$?

We have that the logistic regression predicts $\mathbf{w} \cdot \mathbf{x}_* = 3.1415$.
Then, the probability estimate that $y = 1$ is

$$\mathbb{P}(Y = 1 \mid \mathbf{x}_*) = \frac{1}{1 + e^{-3.1415}} \approx 0.95857 \dots$$

- (d) [harder] What is $\mathcal{H}_{pr}$ for the probability estimation algorithm that employs the linear model in the covariates with cloglog link function?

For the probability estimation algorithm that employs the linear model in the covariates with the cloglog link function we have,

$$\mathcal{H}_{pr} = \left\{ \phi(\mathbf{x}, \mathbf{w}) = 1 - e^{-e^{\mathbf{x} \cdot \mathbf{w}}} : \mathbf{w} \in \mathbb{R}^{p+1} \right\}.$$

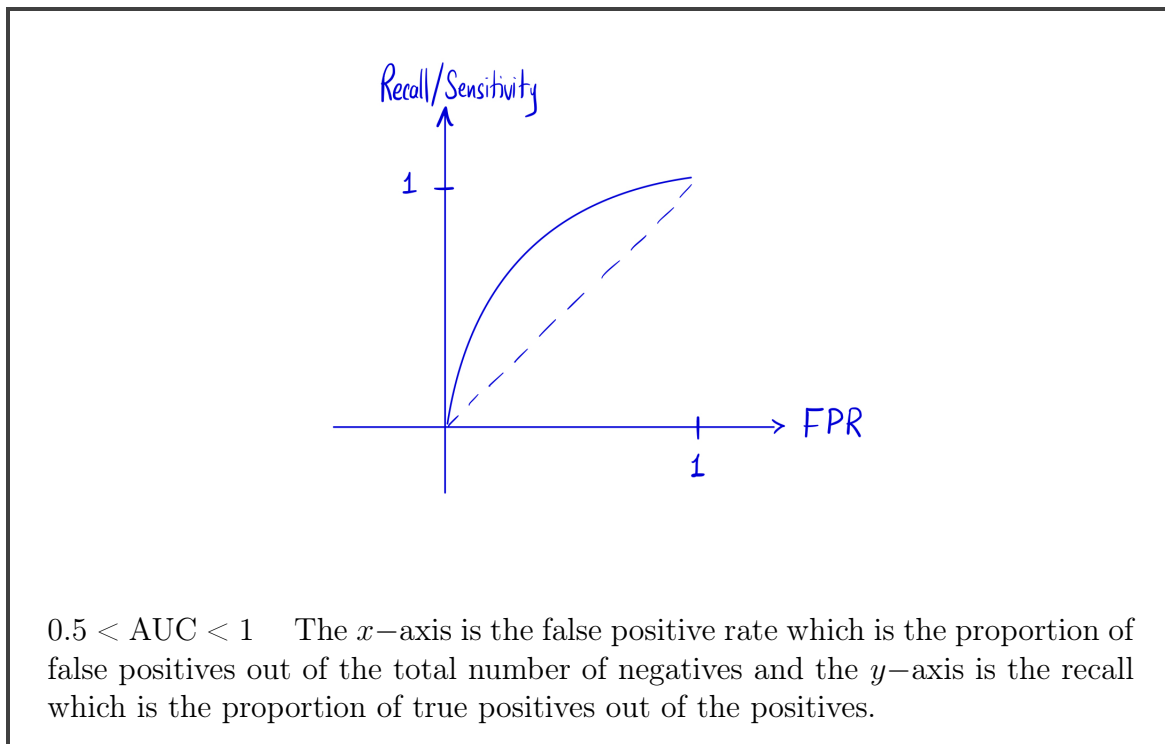
- (e) [difficult] Generalize linear probability estimation to the case where $\mathcal{Y} = \{C_1, C_2, C_3\}$. Use the logistic link function like in logistic regression. Write down the objective function that you would numerically maximize. This objective function is one that is argmax'd over the parameters (you define what these parameters are — that is part of the question).

Once you get the answer you can see how this easily goes to $K > 3$ response categories. The algorithm for general K is known as “multinomial logistic regression”, “polytomous LR”, “multiclass LR”, “softmax regression”, “multinomial logit” (mlogit), the “maximum entropy” (MaxEnt) classifier, and the “conditional maximum entropy model”. You can inflate your resume with lots of jazz by doing this one question!

$$L = - \left(\sum_{i=1}^3 \sum_{k=0}^1 \mathbb{1}_{y^i=k} \log(\mathbb{P}(y^i = k | \mathbf{x}^i; \theta)) \right)$$

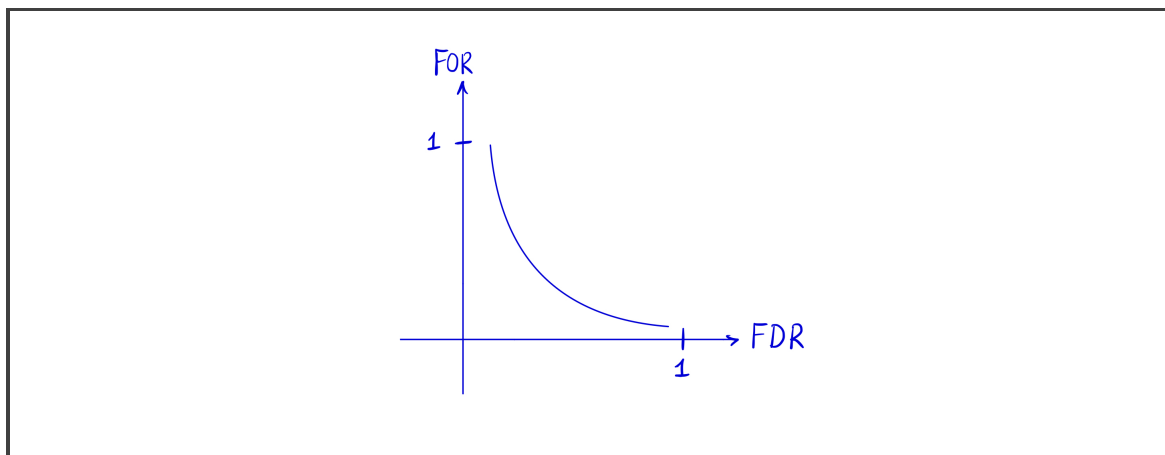
$$\text{where } \mathbb{P}(y^i = k | \mathbf{x}^i; \theta) = \frac{e^{\theta^k \mathbf{x}^i}}{\sum_{j=1}^3 e^{\theta^j \mathbf{x}^i}}$$

- (f) [easy] Graph a canonical ROC and label the axes. In your drawing estimate AUC. Explain very clearly what is measured by the x axis and the y axis.



- (g) [easy] Pick one point on your ROC curve from the previous question. Explain a situation why you would employ this model.

- (h) [harder] Graph a canonical DET curve and label the axes. Explain very clearly what is measured by the x axis and the y axis. Make sure the DET curve's intersections with the axes is correct.



- (i) [easy] Pick one point on your DET curve from the previous question. Explain a situation why you would employ this model.

- (j) [difficult] [MA] The line of random guessing on the ROC curve is the diagonal line with slope one extending from the origin. What is the corresponding line of random guessing in the DET curve? This is not easy...

Problem 2

These are some questions related to bias-variance decomposition. Assume the two assumptions from the notes about the random variable model that produces the δ values, the error due to ignorance.

- (a) [easy] Write down (do not derive) the decomposition of MSE for a given $\mathbf{x}_*$ where $\mathbb{D}$ is assumed fixed but the response associated with $\mathbf{x}_*$ is assumed random.

$$\text{MSE}(\mathbf{x}_*) = [\text{Bias}(\mathbf{x}_*)]^2 + \sigma^2$$

- (b) [easy] Write down (do not derive) the decomposition of MSE for a given $\mathbf{x}_*$ where the responses in $\mathbb{D}$ is random but the $\mathbf{X}$ matrix is assumed fixed and the response associated with $\mathbf{x}_*$ is assumed random like previously.

$$\text{MSE}(\mathbf{x}_*) = \text{Bias}[g(\mathbf{x}_*)]^2 + \mathbb{V}\text{ar}[g(\mathbf{x}_*)] + \sigma^2$$

- (c) [easy] Write down (do not derive) the decomposition of MSE for general predictions of a phenomenon where all quantities are considered random.

$$\text{MSE} = \mathbb{E}[\text{Bias}[g(\mathbf{x}_*)]^2] + \mathbb{E}[\mathbb{V}\text{ar}[g(\mathbf{x}_*)]] + \sigma^2$$

- (d) [difficult] Why is it in (a) there is only a “bias” but no “variance” term? Why did the additional source of randomness in (b) spawn the variance term, a new source of error?

In part (a) there is only a “bias” and no variance term because we assumed that $\mathbb{D}$ was fixed so $g(\mathbf{x}_*)$ is deterministic. However, once $\mathbb{D}$ becomes random then $\mathbf{y}$ becomes random as well. With this additional source of randomness variation of the final hypothesis function to the average function becomes a component of the overall MSE.

- (e) [harder] A high bias / low variance algorithm is underfit or overfit?

A high bias / low variance algorithm tends to underfit.

- (f) [harder] A low bias / high variance algorithm is underfit or overfit?

A low bias / high variance algorithm tends to overfit.

- (g) [harder] Explain why bagging reduces MSE for “free” regardless of the algorithm employed.

Bagging reduces MSE for “free” regardless of the algorithm employed because by looking at the bias-variance decomposition

$$\text{MSE} = \mathbb{E} [\text{Bias}[g(\mathbf{x}_*)]^2] + \rho \mathbb{E} [\text{Var} [g_1(\mathbf{x}_*)]] + \frac{1 - \rho}{M} \mathbb{E} [\text{Var} [g_1(\mathbf{x}_*)]] + \sigma^2$$

and by having the computing power to let $M \rightarrow \infty$ we have a reduction in the variance term of the MSE.

- (h) [harder] Explain why RF reduces MSE atop bagging M trees and specifically mention the target that it attacks in the MSE decomposition formula and why it’s able to reduce that target.

The reason why RF reduces MSE atop bagging M trees is because in the resulting MSE expression for bagging as $M \rightarrow \infty$

$$\text{MSE} = \mathbb{E} [\text{Bias}[g(\mathbf{x}_*)]^2] + \rho \mathbb{E} [\text{Var} [g_1(\mathbf{x}_*)]] + \sigma^2$$

ρ decreases since random forest decorrelates the tree models even, i.e. ρ decreases, since it chooses a random subset of the p_{raw} features of size m_{try} .

- (i) [difficult] When can RF lose to bagging M trees? Hint: think hyperparameter choice.

Random Forest (RF) loses to bagging M trees when the depth of the trees of the random forest is too deep which causes it to overfit the training data and decrease in oos predictive accuracy.

Problem 3

These are some questions related to missingness.

- (a) [easy] [MA] What are the three missing data mechanisms? Provide an example when each occurs (i.e., a real world situation). We didn't really cover this in class so I'm making it a MA question only. This concept will NOT be on the exam.

Missing Data Mechanisms

- (1) Missing Completely at Random (MCAR)
Ex: Some participants may have missing laboratory values because a batch of lab samples was processed improperly. When data are MCAR, the data which remain can be considered a simple random sample of the full data set of interest. MCAR is generally regarded as a strong and often unrealistic assumptions.
- (2) Missing at Random (MAR)
Ex: A registry examining depression may encounter data that are MAR if male participants are less likely to complete a survey about depression severity than female participants. That is, if probability of completion of the survey is related to their sex (which is fully observed) but not the severity of their depression, then the data may be regarded as MAR. When data are MAR, the fact that the data are missing is systematically related to the observed but not the unobserved data.
- (3) Not Missing at Random (NMAR)
Ex: A depression registry may encounter data that are NMAR if participants with severe depression are more likely to refuse to complete the survey about depression severity. When data are NMAR, the fact that the data are missing is systematically related to the unobserved data, that is, the missingness is related to events or factors which are not measured by the researcher.

- (b) [easy] Why is listwise-deletion a *terrible* idea to employ in your $\mathbb{D}$ when doing supervised learning?

The reason for which listwise-deletion is a *terrible* idea to employ in the training data $\mathbb{D}$ when performing supervised learning is because it would increase estimation error due to having less n data. If the number of rows with missing data is very small (approximately 1%) then it is usually permissible, otherwise, it would make the training data $\mathbf{D}$ not representative of the universe of observations.

- (c) [easy] Why is it good practice to augment $\mathbb{D}$ to include missingness dummies? In other words, why would this increase oos predictive accuracy?

The reason why it is good practice to augment $\mathbb{D}$ to include missingness dummies and why it would increase oos predictive accuracy is because missingness in itself can be considered as a feature/attribute of the dataset.

(d) [easy] To impute missing values in $\mathbb{D}$, what is a good default strategy and why?

A good default strategy to impute missing values in $\mathbb{D}$ which starts at $\bar{x}_{.j}$ or $\text{Mode}[\bar{x}_{.j}]$ and iteratively fits random forest models to impute values until the imputed values don't change too much.

Problem 4

These are some questions related to gradient boosting. The final gradient boosted model after M iterations is denoted G_M which can be written in a number of equivalent ways (see below). The g_t 's denote constituent models and the G_t 's denote partial sums of the constituent models up to iteration number t . The constituent models are “steps in functional steps” which have a step size η and a direction component denoted $\tilde{g}_t$. The directional component is the base learner $\mathcal{A}$ fit to the negative gradient of the objective function L which measures how close the current predictions are to the real values of the responses:

$$\begin{aligned}
 G_M &= G_{M-1} + g_M \\
 &= g_0 + g_1 + \dots + g_M \\
 &= g_0 + \eta \tilde{g}_1 + \dots + \eta \tilde{g}_M \\
 &= g_0 + \eta \mathcal{A}(\langle \mathbf{X}, -\nabla L(\mathbf{y}, \hat{\mathbf{y}}_1) \rangle, \mathcal{H}) + \dots + \eta \mathcal{A}(\langle \mathbf{X}, -\nabla L(\mathbf{y}, \hat{\mathbf{y}}_M) \rangle, \mathcal{H}) \\
 &= g_0 + \eta \mathcal{A}(\langle \mathbf{X}, -\nabla L(\mathbf{y}, g_1(\mathbf{X})) \rangle, \mathcal{H}) + \dots + \eta \mathcal{A}(\langle \mathbf{X}, -\nabla L(\mathbf{y}, g_M(\mathbf{X})) \rangle, \mathcal{H})
 \end{aligned}$$

- (a) [easy] From a perspective of only multivariable calculus, explain gradient descent and why it's a good idea to find the minimum inputs for an objective function L (in English).

In the multivariable calculus prospective, we use the algorithm called gradient descent to want to find the minimum of a given function $f : \mathbb{R}^p \rightarrow \mathbb{R}$. We start at a particular initial point $\mathbf{x}_0$. Then we step into a multiple of the negative derivative ∇L . Consider fitting $\hat{\mathbf{y}}$ to $\mathbf{y}$ in a dataset $\mathbb{D}$.

<u>gradient descent</u> $f(\mathbf{x})$ $\mathbf{x}_0$ $\vec{\nabla} f(\mathbf{x})$ η	<u>fitting a response vector</u> objective function $L(\mathbf{y}, \hat{\mathbf{y}})$ starting prediction $\hat{\mathbf{y}}_0$ $\vec{\nabla} L(\mathbf{y}, \hat{\mathbf{y}}) = \begin{bmatrix} \frac{\partial L}{\partial \hat{y}_1} \\ \vdots \\ \frac{\partial L}{\partial \hat{y}_n} \end{bmatrix}$ η
--	---

The iterations look like the following:

$$\begin{aligned}
 \hat{y}_1 &= \hat{y}_0 - \eta \vec{\nabla} L(\mathbf{y}, \hat{\mathbf{y}}_0) \\
 \hat{y}_2 &= \hat{y}_1 - \eta \vec{\nabla} L(\mathbf{y}, \hat{\mathbf{y}}_1) \\
 &\vdots \\
 \hat{y}_M &= \hat{y}_{M-1} - \eta \vec{\nabla} L(\mathbf{y}, \hat{\mathbf{y}}_{M-1}) \approx \hat{\mathbf{y}}_* \approx \mathbf{y} \text{ for all choices of } L
 \end{aligned}$$

- (b) [easy] Write the mathematical steps of gradient boosting for supervised learning below. Use L for the objective function to keep the procedure general. Use notation found in the problem header.

Gradient boosting for supervised learning is applied in much the same vein as gradient descent was outlined in part (a) in the multivariable calculus prospective.

The difference is that the goal of supervised learning is to generalize:

$g = \mathcal{A}(\langle \mathbf{X}, \mathbf{y} \rangle, \mathcal{H})$ where $g : \mathbb{R}^p \rightarrow \mathbb{R}$ which generalizes well and performs accurately for future $\mathbf{x}_*$'s. Also, instead of fitting a model from inputs to response, you fit a model from inputs to the gradient of the objective function. Here are the iterative steps,

$$\begin{aligned} g_0 \\ g_1 &= g_0 + \eta \mathcal{A}(\langle \mathbf{X}, -\nabla L(\mathbf{y}, \hat{\mathbf{y}}_0) \rangle, \mathcal{H}), \quad \hat{\mathbf{y}}_0 = g_0(x) \\ g_2 &= g_1 + \eta \mathcal{A}(\langle \mathbf{X}, -\nabla L(\mathbf{y}, \hat{\mathbf{y}}_1) \rangle, \mathcal{H}), \quad \hat{\mathbf{y}}_1 = g_0(x) + g_1(x) \\ &\vdots \\ g_M &= g_{M-1} + \eta \mathcal{A}(\langle \mathbf{X}, -\nabla L(\mathbf{y}, \hat{\mathbf{y}}_{M-1}) \rangle, \mathcal{H}) \quad \hat{\mathbf{y}}_{M-1} = g_0(x) + \cdots + g_{M-1}(x) \end{aligned}$$

- (c) [easy] For regression, what is $g_0(\mathbf{x})$?

$$g_0(\mathbf{x}) = \bar{y}$$

- (d) [easy] For probability estimation for binary response, what is $g_0(\mathbf{x})$?

$$g_0(\mathbf{x}) = \text{Mode}[y]$$

- (e) [harder] What are all the hyperparameters of gradient boosting? There are more than just two.

Hyperparameters of gradient boosting

- (a) η : learning rate
- (b) M : # of base learners
- (c) N_0 : nodesize (if using trees)
- (d) $\mathcal{A}$: algorithm
- (e) L : the objective functions

- (f) [easy] For regression, rederive the negative gradient of the objective function L .

For regression consider $L(\mathbf{y}, \hat{\mathbf{y}}) = \text{SSE} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$

$$\implies -\vec{\nabla} L(\mathbf{y}, \hat{\mathbf{y}}) = \begin{bmatrix} 2(y_1 - \hat{y}_1) \\ 2(y_2 - \hat{y}_2) \\ \vdots \\ 2(y_n - \hat{y}_n) \end{bmatrix} = 2\mathbf{e}$$

fit $\mathcal{A}(\langle \mathbf{X}, 2\mathbf{e} \rangle, \mathcal{H})$, fitting regression trees to $\mathbf{e}$.

- (g) [easy] For probability estimation for binary response, rederive the negative gradient of the objective function L .

Recall the likelihood function $\prod_{i=1}^n \hat{p}_i^{y_i} (1 - \hat{p}_i)^{1-y_i}$.

Now, reparametrize with log-odds. Let, $\hat{y}_i = \ln \left(\frac{\hat{p}_i}{1 - \hat{p}_i} \right) \iff \hat{p}_i = \frac{e^{\hat{y}_i}}{1 + e^{\hat{y}_i}}$.

The likelihood function then becomes

$$\prod_{i=1}^n \left(\frac{e^{\hat{y}_i}}{1 + e^{\hat{y}_i}} \right)^{y_i} \left(\frac{1}{1 + e^{\hat{y}_i}} \right)^{1-y_i} = \prod_{i=1}^n \frac{e^{\hat{y}_i y_i}}{1 + e^{\hat{y}_i}}$$

Since maximizing the likelihood would yield the same answer as maximizing its log, we then have

$$\ln \left(\prod_{i=1}^n \frac{e^{\hat{y}_i y_i}}{1 + e^{\hat{y}_i}} \right) = \sum_{i=1}^n \left(\frac{e^{\hat{y}_i y_i}}{1 + e^{\hat{y}_i}} \right) = \sum_{i=1}^n \hat{y}_i y_i - \ln (1 + e^{\hat{y}_i})$$

We wish to minimize objective functions in the gradient descent so,

$$L(\mathbf{y}, \hat{\mathbf{y}}) = \sum_{i=1}^n -y_i \hat{y}_i + \ln (1 + e^{\hat{y}_i})$$

and,

$$-\vec{\nabla} L(\mathbf{y}, \hat{\mathbf{y}}) = \begin{bmatrix} y_1 - \frac{e^{\hat{y}_1}}{1 + e^{\hat{y}_1}} \\ y_2 - \frac{e^{\hat{y}_2}}{1 + e^{\hat{y}_2}} \\ \vdots \\ y_n - \frac{e^{\hat{y}_n}}{1 + e^{\hat{y}_n}} \end{bmatrix} = \mathbf{y} - \frac{e^{\hat{\mathbf{y}}}}{1 + e^{\hat{\mathbf{y}}}} = \mathbf{y} - \hat{\mathbf{p}}$$

- (h) [difficult] For probability estimation for binary response scenarios, what is the unit of the output $G_M(\mathbf{x}_\star)$?

The unit of output of $G_M(\mathbf{x}_\star)$ is log-odds.

- (i) [easy] For the base learner algorithm $\mathcal{A}$, why is it a good idea to use shallow CART (which is the recommended default)?

It is a good idea to use shallow CART for the base learner algorithm $\mathcal{A}$ because they are weak learners (high bias, low variance) that can still span function space $g : \mathbb{R}^p \rightarrow \mathbb{R}$ which means it allows for non-linearities and interactions among the p features.

- (j) [difficult] For the base learner algorithm $\mathcal{A}$, why is it a bad idea to use deep CART?

It is a bad idea to use deep CART for the base learner algorithm $\mathcal{A}$ because nodesize N_0 is low which leads to overfitting. We want the base learners to be weak instead.

- (k) [difficult] For the base learner algorithm $\mathcal{A}$, why is it a bad idea to use OLS for regression (or logistic regression for probability estimation for binary response)?

It is a bad idea to use OLS for regression for the base learner algorithm $\mathcal{A}$ because the linear functions from OLS do not span all of function space and therefore are not universal learners and we want functions that can identify nonlinearities and interactions.

- (l) [difficult] If M is very, very large, what is the risk in using gradient boosting even using shallow CART as the base learner (the recommended default)?

If M is very, very larger, then we have the risk of overfitting when implementing gradient boosting even when using the recommended default of shallow CART as the base learner.

- (m) [difficult] If η is very, very large but M reasonably correctly chosen, what is the risk in using gradient boosting even using shallow CART as the base learner (the recommended default)?

If η is very, very large, we may run the risk of using gradient boost and not have the algorithm converge or that our objective function is minimized at an undesirable local minimum.